

Software Implementation and Testing Document

For

Group Hoo's Fishing

Version 1.0

Authors:

Juan Pablo Pastrana II

Tyler Dussault

Bridget Gizzi

Darius Davis

Dhruv Patel

1. Programming Languages (5 points)

GDScript is the main programming language used for Hoo's Fishing and is used for the game's scripts and gameplay systems. The main reason we chose GDScript is that GDScript is designed to work directly with Godot, making it straightforward to connect scripts with Godot features such as scenes, nodes, signals and resources.

2. Platforms, APIs, Databases, and other technologies used (5 points)

Godot 4 is the main platform used to build the game. It is used to create the scenes, user interface, resources, and gameplay systems. The project also uses Godot resources to store game data such as fish and equipment information, but no external database is currently required because the game data is stored locally within the Godot project.

3. Execution-based Functional Testing (10 points)

We perform functional testing by running the game in Godot and checking that each gameplay system works as intended before changes are merged into the main project. Individual features are tested first on their own, such as player movement, fishing, fish spawning, inventory and hotbar behavior, selling fish, and the money system. For example, one of our functional requirements is that the player should be able to catch a fish and have it added to the inventory , so we test the full process from starting the fishing interaction to confirming that the caught fish appears correctly in the hotbar. After individual features are working, we perform integration testing by combining changes with the rest of the project and checking that connected systems still work together correctly. This is important for systems such as fishing, inventory, the shop, and money, since they depend on each other as part of the main gameplay loop.

4. Execution-based Non-Functional Testing (10 points)

We perform non-functional testing by running the main game scene and using feedback from team members on what works well and what needs improvement. Based on feedback about the game's performance and visual appearance, we make changes as needed.

5. Non-Execution-based Testing (10 points)

We organize non-execution-based testing through GitHub, our group chat, and team meetings. Team members review committed code and project changes, and during meetings we walk through newly added code and discuss how the new code and systems connect with the existing project. Commits and pull requests are also documented so other team members can clearly see what is being added or changed. We review changes before and after they are merged into the main project to check for conflicts. This helps us understand each other's work, catch issues earlier, and make sure new features fit the existing project structure before continuing.